PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL)

Complete Report — 10 Selected Case Studies

Institution	Dr. RVR NRI Institute of Technology
Programme	B. Tech – CSM (AI & ML)
Student Name	D. SAI SRIVARSHA
Roll Number	26001068
Branch / Section	CSM / 4
Course	Python Programming
GitHub Repository	PythonActivity-Based-Learning

This report contains the required documentation for each selected case study, together with the output screenshots taken after running the corresponding Python file.

Report Guide

Mandatory sections included for every case: Problem Statement, Objectives, Assumptions, Algorithm, Flowchart, Pseudocode, Python Source Code, Test Cases, Output Screenshot(s), and Conclusion.

Selected case studies

Case	Case Study	Python file
2	Progressive Electricity Billing	02_Electricity_Billing.py
7	Retail Billing with Multiple Discount Rules	07_Retail_Billing.py
27	Advanced Numeric Pattern Generator	27_Numeric_Pattern.py
29	Password Policy Validation Engine	29_Password_Validation.py
35	Text Compression Using Character Runs	35_Text_Compression.py
37	Nested Loop Calendar Generator	37_Calendar_Generator.py
42	Student Marks Matrix Analysis	42_Marks_Matrix.py
43	Multi-Branch Sales Matrix Comparison	43_Sales_Matrix.py
48	Tuple-Based Employee Shift Conflict Detection	48_Shift_Conflicts.py
49	Tuple-Based Product Catalogue Audit	49_Product_Audit.py

Case 2: Progressive Electricity Billing

Python file: **02_Electricity_Billing.py**

1. Problem Statement

Calculate the monthly electricity bill of a household using consumption slabs, a fixed charge, a surcharge and a rebate. Compare the bills of several months and identify any unusual increase.

2. Objectives

- Calculate the energy charge using slab rates.
- Apply the fixed charge, surcharge and rebate.
- Store the units and bills of multiple months in lists.
- Compare months and detect unusual increases.

3. Assumptions

- Slab 1: first 100 units at Rs 3 per unit; slab 2: next 100 units at Rs 5; above 200 units at Rs 8.
- Fixed charge is Rs 50 per month.
- Surcharge of 10% of the energy charge if units are more than 300.
- Rebate of 5% of the energy charge if units are less than 50.
- A bill that is 40% or more higher than the previous month is an unusual increase.
- Units must be zero or more (whole numbers).

4. Algorithm

1. Start.
2. Read the number of months n.
3. Create two empty lists: units_list and bills_list.
4. For each month, read units; if units are negative, show an error and read again.
5. Calculate the energy charge using the three slabs.
6. If units > 300 add a 10% surcharge; if units < 50 subtract a 5% rebate.
7. Bill = energy + surcharge - rebate + fixed charge; store units and bill in the lists.
8. Print the table of months, units and bills.
9. For each month from the 2nd, find the percentage increase over the previous bill.
10. If the increase is 40% or more, print it as an unusual increase; otherwise print None.
11. Stop.

5. Flowchart

START → Read months → Repeat: read and validate units → Slab calculation → Apply surcharge / rebate → Add fixed charge and store bill → Print table → Compare months (≥ 40% increase?) → Print result → STOP

6. Pseudocode

```
1. Read n; create units_list and bills_list.
2. For each month: read units until units >= 0.
3. If units <= 100: energy = units * 3; else if units <= 200: energy = 300 + (units - 100) * 5;
   else energy = 800 + (units - 200) * 8.
4. If units > 300: surcharge = 10% of energy.  If units < 50: rebate = 5% of energy.
5. bill = energy + surcharge - rebate + 50; append units and bill to the lists.
6. Print the table. For each month from the 2nd: increase = (bill - previous bill) / previous bill * 100.
7. If increase >= 40 print 'Unusual increase'; if none found print 'None'.
```

7. Python Source Code

```
# Progressive Electricity Billing

FIXED_CHARGE = 50

n = int(input("Enter number of months: "))
```

```
units_list = []
bills_list = []

for i in range(n):
    # input validation
    while True:
        units = int(input("Enter units for month " + str(i + 1) + ": "))
        if units >= 0:
            break
        print("Invalid! Units cannot be negative.")

    # slab calculation
    if units <= 100:
        energy = units * 3
    elif units <= 200:
        energy = 100 * 3 + (units - 100) * 5
    else:
        energy = 100 * 3 + 100 * 5 + (units - 200) * 8

    # surcharge and rebate
    surcharge = 0
    rebate = 0
    if units > 300:
        surcharge = 0.10 * energy
    if units < 50:
        rebate = 0.05 * energy

    # final bill
    bill = energy + surcharge - rebate + FIXED_CHARGE

    units_list.append(units)
    bills_list.append(bill)

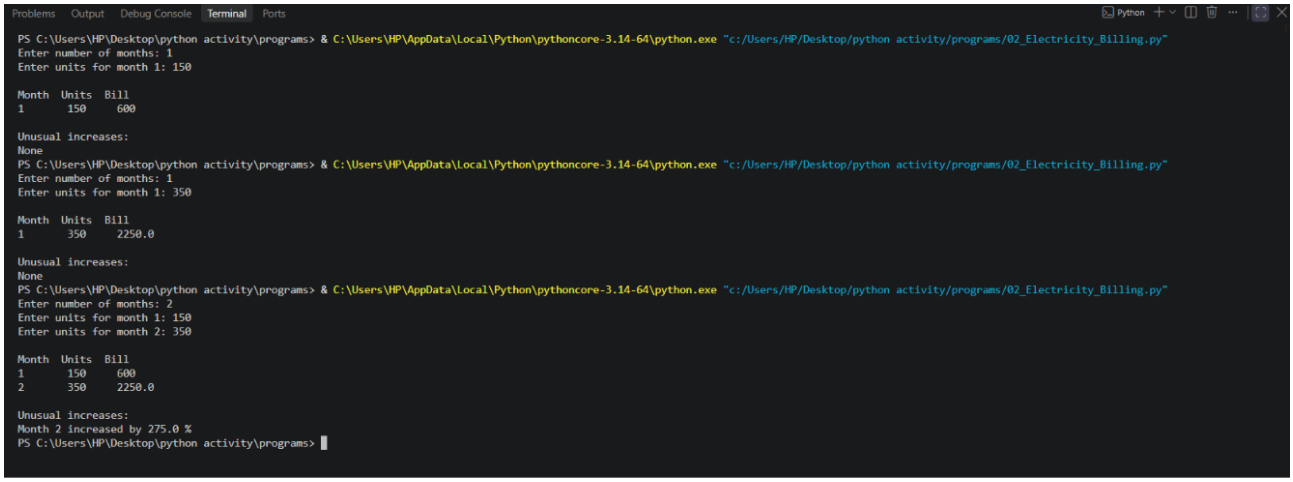
# summary
print("\nMonth  Units  Bill")
for i in range(n):
    print(i + 1, "     ", units_list[i], "   ", round(bills_list[i], 2))

# unusual increase check (40% or more)
print("\nUnusual increases:")
found = False
for i in range(1, n):
    increase = (bills_list[i] - bills_list[i - 1]) / bills_list[i - 1] * 100
    if increase >= 40:
        print("Month", i + 1, "increased by", round(increase, 2), "%")
        found = True
if not found:
    print("None")
```

8. Test Cases

- Test 1: 1 month, 150 units -> bill 600; no unusual increase
- Test 2: 1 month, 350 units -> bill 2250.0 (surcharge applied); no unusual increase
- Test 3: 2 months, 150 and 350 units -> bills 600 and 2250.0; Month 2 increased by 275.0 %

9. Output Screenshot(s)



10. Conclusion

The program calculated the electricity bill correctly using slabs, a fixed charge, a surcharge and a rebate. Lists stored the bills of many months and loops with conditions detected unusual increases.

Case 7: Retail Billing with Multiple Discount Rules

Python file: **07_Retail_Billing.py**

1. Problem Statement

Generate an itemised retail invoice for several products with category-wise tax, a threshold-based discount on the subtotal, and a maximum discount limit.

2. Objectives

• Read product details and validate price, quantity and category.
• Apply category-wise tax rates.
• Apply the threshold discount with a maximum limit.
• Print an itemised invoice with the final bill.

3. Assumptions

• Tax rates: Grocery (G) 5%, Clothing (C) 12%, Electronics (E) 18%.
• Discount on subtotal: 5% above Rs 2000 and 10% above Rs 5000.
• The maximum discount is Rs 500.
• Price and quantity must be positive.
• Category must be G, C or E (small or capital letters).

4. Algorithm

1. Start.
2. Read the number of products n.
3. For each product read name, price, quantity and category; validate and read again if invalid.
4. Find the tax rate from the category, then amount = price * quantity and tax = amount * rate.
5. Store the details in lists.
6. Subtotal = sum of amounts; total tax = sum of taxes.
7. Discount = 10% if subtotal > 5000, 5% if subtotal > 2000, otherwise 0.
8. If discount > 500, set discount = 500 and mark it as capped.
9. Final bill = subtotal - discount + total tax.
10. Print the invoice and stop.

5. Flowchart

START → Read products → Validate price / quantity / category → Calculate amount and tax → Subtotal and total tax → Discount slab → Cap at Rs 500? → Final bill → Print invoice → STOP

6. Pseudocode

```
1. Read n. For each product read name, price, quantity, category (repeat until valid).
2. If category = G rate = 0.05; else if C rate = 0.12; else rate = 0.18.
3. amount = price * quantity; tax = amount * rate; store in lists.
4. subtotal = sum(amounts); total_tax = sum(taxes).
5. If subtotal > 5000 discount = 10%; else if subtotal > 2000 discount = 5%; else 0.
6. If discount > 500 then discount = 500 (capped).
7. final_bill = subtotal - discount + total_tax; print the invoice.
```

7. Python Source Code

```
# Retail Billing with Multiple Discount Rules

MAX_DISCOUNT = 500

n = int(input("Enter number of products: "))
names = []
qtys = []
```

```
prices = []
amounts = []
taxes = []

for i in range(n):
    name = input("Enter product name: ")

    # input validation
    while True:
        price = float(input("Enter price: "))
        qty = int(input("Enter quantity: "))
        cat = input("Enter category (G/C/E): ").upper()
        if price > 0 and qty > 0 and cat in ["G", "C", "E"]:
            break
        print("Invalid! Price, quantity must be positive and category must be G, C or E.")

    # category-wise tax rate
    if cat == "G":
        rate = 0.05
    elif cat == "C":
        rate = 0.12
    else:
        rate = 0.18

    amount = price * qty
    tax = amount * rate

    names.append(name)
    qtys.append(qty)
    prices.append(price)
    amounts.append(amount)
    taxes.append(tax)

subtotal = sum(amounts)
total_tax = sum(taxes)

# threshold discount
if subtotal > 5000:
    discount = subtotal * 0.10
elif subtotal > 2000:
    discount = subtotal * 0.05
else:
    discount = 0

# maximum concession limit
capped = False
if discount > MAX_DISCOUNT:
    discount = MAX_DISCOUNT
    capped = True

final_bill = subtotal - discount + total_tax

# itemized invoice
print("\n----- INVOICE -----")
print("Item   Qty   Price   Amount   Tax")
for i in range(n):
    print(names[i], "  ", qtys[i], "  ", prices[i], "  ", round(amounts[i], 2), "  ", round(taxes[i], 2))
print("--------------------")
print("Subtotal:", round(subtotal, 2))
print("Discount:", round(discount, 2))
if capped:
    print("(Discount capped at Rs 500)")
print("Total Tax:", round(total_tax, 2))
print("Final Bill:", round(final_bill, 2))
```

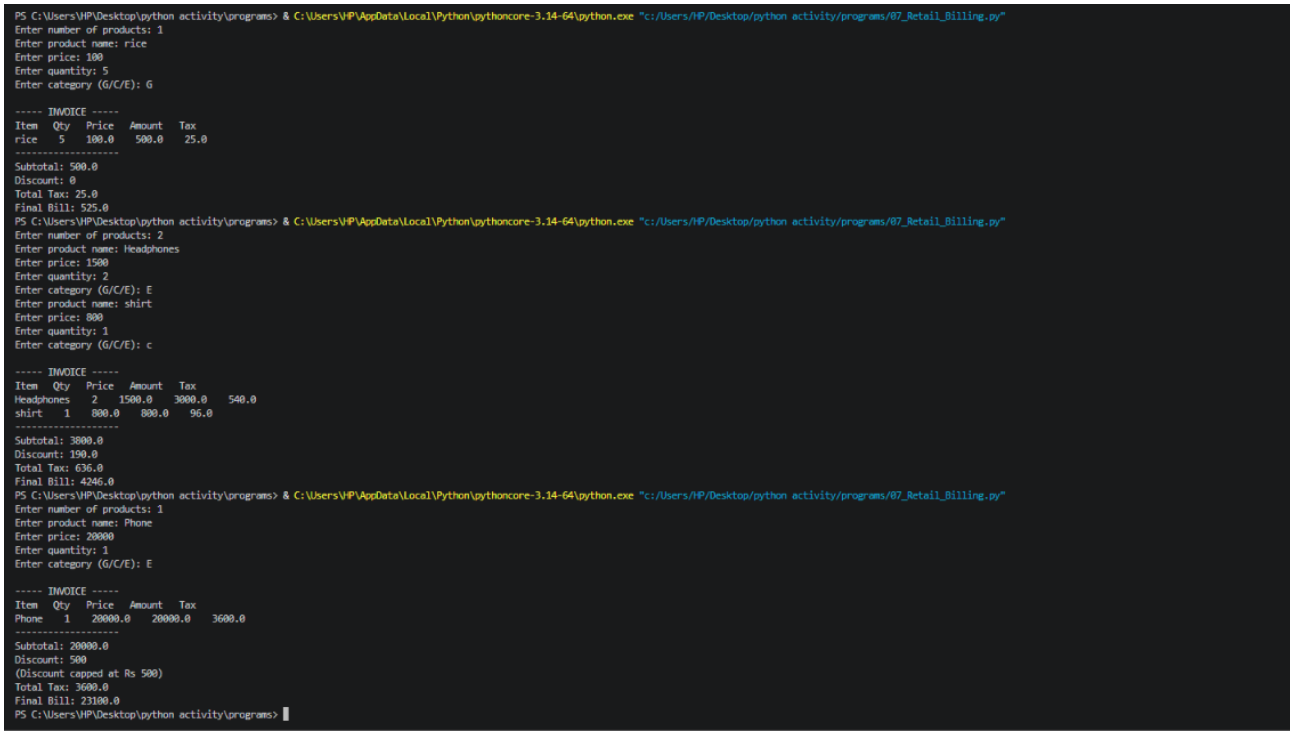
8. Test Cases

• Test 1: rice, price 100, qty 5, G -> subtotal 500.0; discount 0; tax 25.0; final bill 525.0

• Test 2: Headphones 1500 x 2 (E) and shirt 800 x 1 (C) -> subtotal 3800.0; discount 190.0; tax 636.0; final bill 4246.0

• Test 3: Phone, price 20000, qty 1, E -> discount capped at Rs 500; tax 3600.0; final bill 23100.0

9. Output Screenshot(s)



10. Conclusion

The program produced an itemised invoice with correct tax, a threshold discount and the Rs 500 cap. Lists, loops and conditionals were used to process any number of products.

Case 27: Advanced Numeric Pattern Generator

Python file: **27_Numeric_Pattern.py**

1. Problem Statement

Print a hollow diamond pattern of a given odd size where the border contains numbers and the inside is empty.

2. Objectives

• Use nested loops and string handling to print a pattern.
• Calculate leading spaces and the inner gap for each row.
• Validate the size entered by the user.

3. Assumptions

• The size n must be an odd number.
• The minimum size is 3.
• The number printed in a row is its distance from the top or bottom tip plus one.

4. Algorithm

1. Start.
2. Read the size n.
3. If n < 3 or n is even, print an error and stop.
4. Find the middle row: mid = n // 2.
5. For each row i, find d = distance from the middle row (i if i <= mid, else n - 1 - i).
6. Leading spaces = mid - d and number = d + 1.
7. If d = 0 print the single number; otherwise print the number, (2d - 1) spaces and the number again.
8. Stop.

5. Flowchart

START → Read n → Valid (odd, ≥ 3)? → If no print error → If yes, for each row find d, spaces, number → Print row → STOP

6. Pseudocode

```
1. Read n. If n < 3 or n is even: print 'Invalid size' and stop.
2. mid = n // 2.
3. For i = 0 to n - 1:
4.    d = i if i <= mid else n - 1 - i
5.    line = (mid - d) spaces
6.    If d = 0: line = line + (d + 1)
7.    Else: line = line + (d + 1) + (2d - 1) spaces + (d + 1)
8.    Print line.
```

7. Python Source Code

```
# Hollow Diamond Numeric Pattern

n = int(input("Enter size (odd number, minimum 3): "))

if n < 3 or n % 2 == 0:
    print("Invalid size! Enter an odd number that is 3 or more.")
else:
    mid = n // 2

    for i in range(n):
        # distance from the middle row
        if i <= mid:
            d = i
        else:
```

```
        d = n - 1 - i

    spaces = mid - d
    number = d + 1

    # leading spaces
    line = " " * spaces

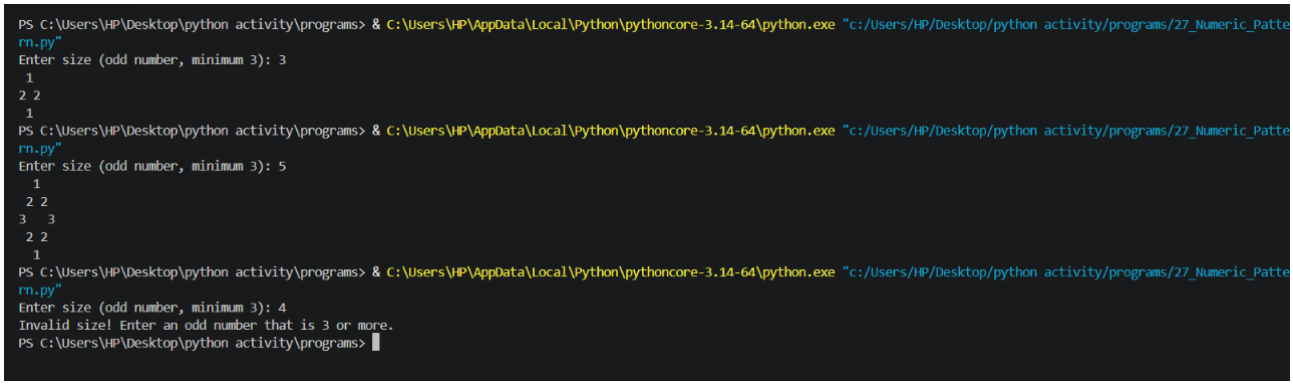
    if d == 0:
        # top and bottom tip
        line = line + str(number)
    else:
        # left number, inner gap, right number
        line = line + str(number) + " " * (2 * d - 1) + str(number)

    print(line)
```

8. Test Cases

- Test 1: n = 3 -> diamond of 3 rows (1 / 2 2 / 1)
- Test 2: n = 5 -> diamond of 5 rows with 3 3 in the middle row
- Test 3: n = 4 -> Invalid size! Enter an odd number that is 3 or more.

9. Output Screenshot(s)



10. Conclusion

The program generated the hollow diamond pattern using nested logic and rejected invalid sizes. It shows how loops and string repetition build patterns.

Case 29: Password Policy Validation Engine

Python file: **29_Password_Validation.py**

1. Problem Statement

Check a password against a set of rules and display every rule that it fails.

2. Objectives

• Check length and character categories using a loop over the string.
• Detect three repeated characters in a row and prohibited words.
• Collect all failed rules in a list and display them.

3. Assumptions

• Minimum length is 8.
• At least one uppercase letter, one lowercase letter, one digit and one special character (!@#$%&*).
• The same character must not repeat 3 times in a row.
• Prohibited words are password, 1234 and qwerty (capital or small).

4. Algorithm

1. Start.
2. Read the password and create an empty list errors.
3. If the length is less than 8, add an error.
4. Check each character and note whether an uppercase, lowercase, digit and special character is present; add errors for missing ones.
5. Check every position for three equal characters in a row; add an error if found.
6. Convert the password to lower case and check for each prohibited word; add an error if found.
7. If errors is empty print VALID; otherwise print INVALID and all reasons.
8. Stop.

5. Flowchart

START → Read password → Check length → Check character types (loop) → Check repetition → Check prohibited words → Errors empty? → Print VALID / INVALID with reasons → STOP

6. Pseudocode

```
1. Read pwd; errors = [].
2. If length(pwd) < 8 add 'Length must be at least 8'.
3. For each ch in pwd: mark has_upper / has_lower / has_digit / has_special.
4. For each missing category add an error.
5. For i = 0 to length - 3: if pwd[i] = pwd[i+1] = pwd[i+2] then repeat = true.
6. For each banned word: if it is in lower(pwd) add an error.
7. If errors is empty print VALID else print INVALID and each error.
```

7. Python Source Code

```
# Password Policy Validation Engine

SPECIALS = "!@#$%&*"
BANNED = ["password", "1234", "qwerty"]

pwd = input("Enter password: ")
errors = []

# rule 1: length
if len(pwd) < 8:
    errors.append("Length must be at least 8")

# rule 2: character categories
has_upper = False
```

```
has_lower = False
has_digit = False
has_special = False

for ch in pwd:
    if ch >= "A" and ch <= "Z":
        has_upper = True
    elif ch >= "a" and ch <= "z":
        has_lower = True
    elif ch >= "0" and ch <= "9":
        has_digit = True
    elif ch in SPECIALS:
        has_special = True

if not has_upper:
    errors.append("Must contain an uppercase letter")
if not has_lower:
    errors.append("Must contain a lowercase letter")
if not has_digit:
    errors.append("Must contain a digit")
if not has_special:
    errors.append("Must contain a special character (!@#$%&*)")

# rule 3: same character 3 times in a row
repeat = False
for i in range(len(pwd) - 2):
    if pwd[i] == pwd[i + 1] and pwd[i + 1] == pwd[i + 2]:
        repeat = True
if repeat:
    errors.append("No character should repeat 3 times in a row")

# rule 4: prohibited words
lower_pwd = pwd.lower()
for word in BANNED:
    if word in lower_pwd:
        errors.append("Contains prohibited word: " + word)

# result
if len(errors) == 0:
    print("Password is VALID")
else:
    print("Password is INVALID")
    print("Reasons:")
    for e in errors:
        print("-", e)
```

8. Test Cases

- Test 1: varsha@2026 -> INVALID; Must contain an uppercase letter
- Test 2: pass -> INVALID; length less than 8, no uppercase letter, no digit, no special character
- Test 3: siri2902 -> INVALID; no uppercase letter, no special character

9. Output Screenshot(s)



10. Conclusion

The program validated passwords against several rules and reported every failed rule together. Strings, loops, conditions and lists were used.

Case 35: Text Compression Using Character Runs

Python file: **35_Text_Compression.py**

1. Problem Statement

Compress a string by replacing each run of repeated characters with the character and its count, then compare the original and encoded lengths.

2. Objectives

• Count consecutive repeated characters using a loop.
• Build the encoded string.
• Decide whether the compression is beneficial.

3. Assumptions

• The input is a non-empty string of letters.
• Compression is beneficial only if the encoded length is less than the original length.

4. Algorithm

1. Start.
2. Read the string; if it is empty print an error and stop.
3. Set encoded = empty and count = 1.
4. For each character from the 2nd, if it equals the previous one increase count.
5. Otherwise add the previous character and count to encoded and reset count to 1.
6. After the loop add the last character and its count.
7. Print the original, the encoded string and both lengths.
8. If the encoded length is less than the original, print that compression is beneficial; otherwise print that it is not.
9. Stop.

5. Flowchart

START → Read string → Empty? → Loop over characters (same as previous? count + 1 : store run) → Add last run → Print lengths → Beneficial? → Print result → STOP

6. Pseudocode

```
1. Read text. If length is 0: print 'Invalid' and stop.
2. encoded = ''; count = 1.
3. For i = 1 to length - 1:
4.    If text[i] = text[i-1]: count = count + 1
5.    Else: encoded = encoded + text[i-1] + count; count = 1
6. encoded = encoded + last character + count.
7. Print original, encoded and lengths.
8. If length(encoded) < length(text) print 'beneficial' else print 'NOT beneficial'.
```

7. Python Source Code

```
# Text Compression Using Character Runs

text = input("Enter a string: ")

if len(text) == 0:
    print("Invalid! String cannot be empty.")
else:
    encoded = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count = count + 1
        else:
```

```
            encoded = encoded + text[i - 1] + str(count)
            count = 1

    # add the last run
    encoded = encoded + text[len(text) - 1] + str(count)

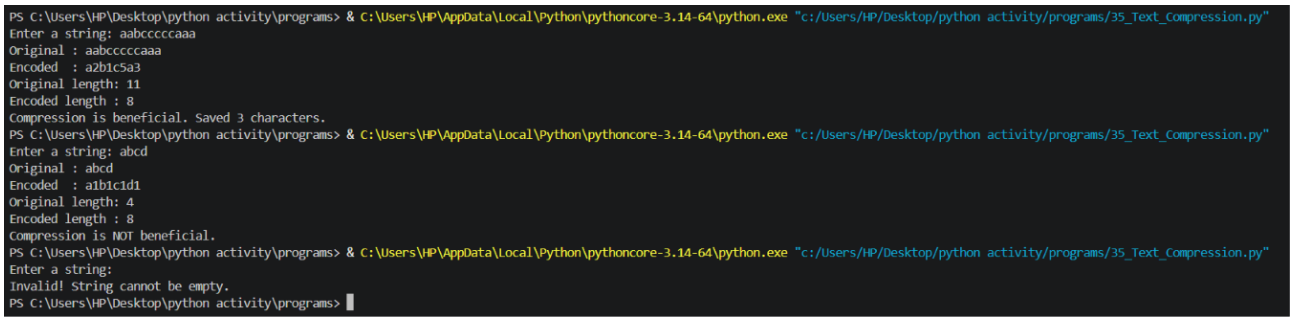
    print("Original :", text)
    print("Encoded  :", encoded)
    print("Original length:", len(text))
    print("Encoded length :", len(encoded))

    if len(encoded) < len(text):
        print("Compression is beneficial. Saved", len(text) - len(encoded), "characters.")
    else:
        print("Compression is NOT beneficial.")
```

8. Test Cases

- Test 1: aabcccccaaa -> a2b1c5a3; lengths 11 and 8; compression beneficial, saved 3 characters
- Test 2: abcd -> a1b1c1d1; lengths 4 and 8; compression NOT beneficial
- Test 3: empty string -> Invalid! String cannot be empty.

9. Output Screenshot(s)



10. Conclusion

The program encoded runs of characters correctly and judged whether compression helps. It shows string traversal and comparison of adjacent characters.

Case 37: Nested Loop Calendar Generator

Python file: **37_Calendar_Generator.py**

1. Problem Statement

Print the calendar of a month in a week-wise table, given the starting weekday and the number of days.

2. Objectives

• Use nested loops to print rows (weeks) and columns (days).
• Leave blank spaces before the first date.
• Validate the weekday and number of days.

3. Assumptions

• Starting weekday: 1 = Monday ... 7 = Sunday.
• Number of days must be between 28 and 31.
• The calendar starts on Monday.

4. Algorithm

1. Start.
2. Read the starting weekday and the number of days.
3. If either value is invalid, print an error and stop.
4. Print the header Mon to Sun and set day = 1.
5. Outer loop: for each week (up to 6), stop if all days are printed.
6. Inner loop: for each column 1 to 7, print a blank if it is before the starting weekday in week 1 or after the last day.
7. Otherwise print the date (right aligned) and increase day.
8. Print each week on its own line.
9. Stop.

5. Flowchart

START → Read start and days → Valid? → Print header → For each week → For each column (blank or date) → Print week → All days printed? → STOP

6. Pseudocode

```
1. Read start and days. If start not in 1..7 or days not in 28..31: print error and stop.
2. Print 'Mon Tue Wed Thu Fri Sat Sun'; day = 1.
3. For week = 0 to 5: if day > days then stop.
4.    line = ''
5.    For col = 1 to 7:
6.       If (week = 0 and col < start) or day > days: add blank to line
7.       Else: add day to line; day = day + 1
8.    Print line.
```

7. Python Source Code

```
# Nested Loop Calendar Generator

start = int(input("Enter starting weekday (1=Mon ... 7=Sun): "))
days = int(input("Enter number of days (28 to 31): "))

if start < 1 or start > 7:
    print("Invalid starting weekday!")
elif days < 28 or days > 31:
    print("Invalid number of days!")
else:
    print("Mon Tue Wed Thu Fri Sat Sun")
    day = 1

    for week in range(6):
        # stop if all days are already printed
```

```
        if day > days:
            break

        line = ""
        for col in range(1, 8):
            # blank before the 1st day, or after the last day
            if (week == 0 and col < start) or day > days:
                line = line + "    "
            else:
                line = line + str(day).rjust(3) + " "
                day = day + 1
        print(line)
```

8. Test Cases

- Test 1: start 3 (Wednesday), 30 days -> calendar starting under Wed, last date 30
- Test 2: start 1 (Monday), 28 days -> four full weeks
- Test 3: start 8, 30 days -> Invalid starting weekday!

9. Output Screenshot(s)



10. Conclusion

The program printed a correctly aligned calendar using nested loops. It shows how rows and columns, blanks and right-aligned numbers create a table.

Case 42: Student Marks Matrix Analysis

Python file: **42_Marks_Matrix.py**

1. Problem Statement

Store the marks of several students in several subjects as a matrix (nested list) and analyse student-wise and subject-wise performance.

2. Objectives

• Create and read a nested list of marks.
• Find the total and average of each student (row-wise).
• Find the top performer(s).
• Find the average and highest mark of each subject (column-wise) and flag low-average subjects.

3. Assumptions

• Marks must be between 0 and 100.
• A subject with an average below 40 is a low-average subject.
• If several students have the same highest total, all are top performers.

4. Algorithm

1. Start.
2. Read the number of students n and subjects m; both must be at least 1.
3. Read the marks into a nested list, validating each mark (0 to 100).
4. For each student (row) find the total and average and store the total in a list.
5. Find the highest total and print every student with that total.
6. For each subject (column) find the average and the highest mark.
7. Add subjects with an average below 40 to a low-subject list.
8. Print the low-average subjects or None.
9. Stop.

5. Flowchart

START → Read n and m → Valid? → Read marks matrix → Row-wise totals and averages → Top performer(s) → Column-wise average and highest → Low-average subjects → STOP

6. Pseudocode

```
1. Read n, m. If n < 1 or m < 1: print error and stop.
2. marks = []; for each student read m marks (0 to 100) into a row; append row.
3. For each row: total = sum of the row; average = total / m; append total to totals.
4. highest = max(totals); print every student whose total = highest.
5. For each column j: col_total = sum of marks[i][j]; avg = col_total / n; col_high = largest value.
6. If avg < 40 add subject j + 1 to low_subjects.
7. Print low_subjects or 'None'.
```

7. Python Source Code

```
# Student Marks Matrix Analysis

LOW_AVG = 40

n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))

if n < 1 or m < 1:
    print("Invalid! Students and subjects must be at least 1.")
else:
    # build the matrix (nested list)
    marks = []
    for i in range(n):
        row = []
        for j in range(m):
            while True:
```

```
            mark = int(input("Enter marks of student " + str(i + 1) + " subject " + str(j + 1) + ": "))
            if mark >= 0 and mark <= 100:
                break
            print("Invalid! Marks must be between 0 and 100.")
        row.append(mark)
    marks.append(row)

# row-wise: total and average of each student
print("\n--- Student-wise ---")
totals = []
for i in range(n):
    total = 0
    for j in range(m):
        total = total + marks[i][j]
    totals.append(total)
    print("Student", i + 1, "Total:", total, "Average:", round(total / m, 2))

# top performer(s)
highest = max(totals)
print("\nTop performer(s) with total", highest, ":")
for i in range(n):
    if totals[i] == highest:
        print("Student", i + 1)

# column-wise: average and highest of each subject
print("\n--- Subject-wise ---")
low_subjects = []
for j in range(m):
    col_total = 0
    col_high = marks[0][j]
    for i in range(n):
        col_total = col_total + marks[i][j]
        if marks[i][j] > col_high:
            col_high = marks[i][j]
    avg = col_total / n
    print("Subject", j + 1, "Average:", round(avg, 2), "Highest:", col_high)
    if avg < LOW_AVG:
        low_subjects.append(j + 1)

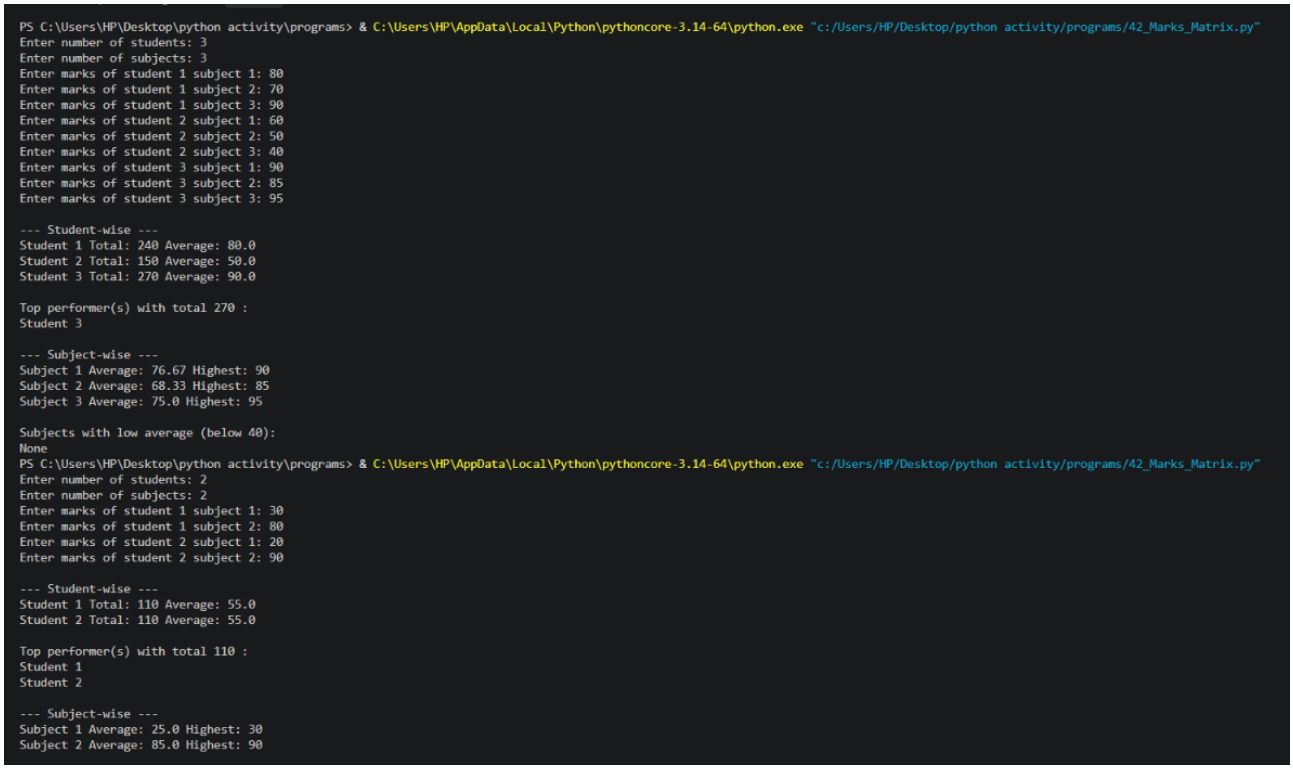
print("\nSubjects with low average (below 40):")
if len(low_subjects) == 0:
    print("None")
else:
    for s in low_subjects:
        print("Subject", s)
```

8. Test Cases

• Test 1: 3 students, 3 subjects, marks 80 70 90 / 60 50 40 / 90 85 95 -> totals 240, 150, 270; top performer Student 3; no low-average subject

• Test 2: 2 students, 2 subjects, marks 30 80 / 20 90 -> both totals 110, so Student 1 and Student 2 are top performers; Subject 1 (average 25.0) is low

9. Output Screenshot(s)



10. Conclusion

The program analysed a marks matrix row-wise and column-wise and handled ties. It shows how nested lists are traversed with nested loops.

Case 43: Multi-Branch Sales Matrix Comparison

Python file: **43_Sales_Matrix.py**

1. Problem Statement

Store the sales of several products in several branches over some days as a matrix and compare branches and products.

2. Objectives

• Build a branch x product matrix from daily sales.
• Find product totals (column-wise) and branch totals (row-wise).
• Find the best branch(es) and the best product in each branch.

3. Assumptions

• Sales are units sold and cannot be negative.
• Daily sales of a branch and product are added to form one matrix entry.
• If branches tie for the highest total, all are printed as best.

4. Algorithm

1. Start.
2. Read the number of branches, products and days; all must be at least 1.
3. For each branch and product read the sales of every day, validate them and add them to a total.
4. Store the total in a nested list (rows = branches, columns = products).
5. Print the matrix.
6. Find the total of each product (column-wise) and of each branch (row-wise).
7. Find the highest branch total and print every branch with that total.
8. For each branch find the product with the highest sales.
9. Stop.

5. Flowchart

START → Read branches, products, days → Valid? → Read daily sales and total them → Print matrix → Product totals → Branch totals → Best branch(es) → Best product per branch → STOP

6. Pseudocode

```
1. Read b, p, d. If any is less than 1: print error and stop.
2. For each branch i and product j: total = sum of d daily sales (each >= 0).
3. sales[i][j] = total.
4. Print each row of sales.
5. Product total j = sum of sales[i][j] over all branches.
6. Branch total i = sum of sales[i][j] over all products.
7. Print every branch whose total = max(branch_totals).
8. For each branch find the product index with the largest sales[i][j].
```

7. Python Source Code

```
# Multi-Branch Sales Matrix Comparison

b = int(input("Enter number of branches: "))
p = int(input("Enter number of products: "))
d = int(input("Enter number of days: "))

if b < 1 or p < 1 or d < 1:
    print("Invalid! Branches, products and days must be at least 1.")
else:
    # build matrix: rows = branches, columns = products
    sales = []
    for i in range(b):
```

```
        row = []
        for j in range(p):
            total = 0
            for k in range(d):
                while True:
                    v = int(input("Branch " + str(i + 1) + " Product " + str(j + 1) + " Day " + str(k + 1) + ": "))
                    if v >= 0:
                        break
                    print("Invalid! Sales cannot be negative.")
                total = total + v
            row.append(total)
        sales.append(row)

    print("\n--- Sales Matrix (rows = branches, columns = products) ---")
    for i in range(b):
        print("Branch", i + 1, ":", sales[i])

    # product totals (column-wise)
    print("\n--- Product totals ---")
    for j in range(p):
        col_total = 0
        for i in range(b):
            col_total = col_total + sales[i][j]
        print("Product", j + 1, "Total:", col_total)

    # branch totals (row-wise)
    print("\n--- Branch totals ---")
    branch_totals = []
    for i in range(b):
        t = 0
        for j in range(p):
            t = t + sales[i][j]
        branch_totals.append(t)
        print("Branch", i + 1, "Total:", t)

    # best branch(es)
    highest = max(branch_totals)
    print("\nBest branch(es) with total", highest, ":")
    for i in range(b):
        if branch_totals[i] == highest:
            print("Branch", i + 1)

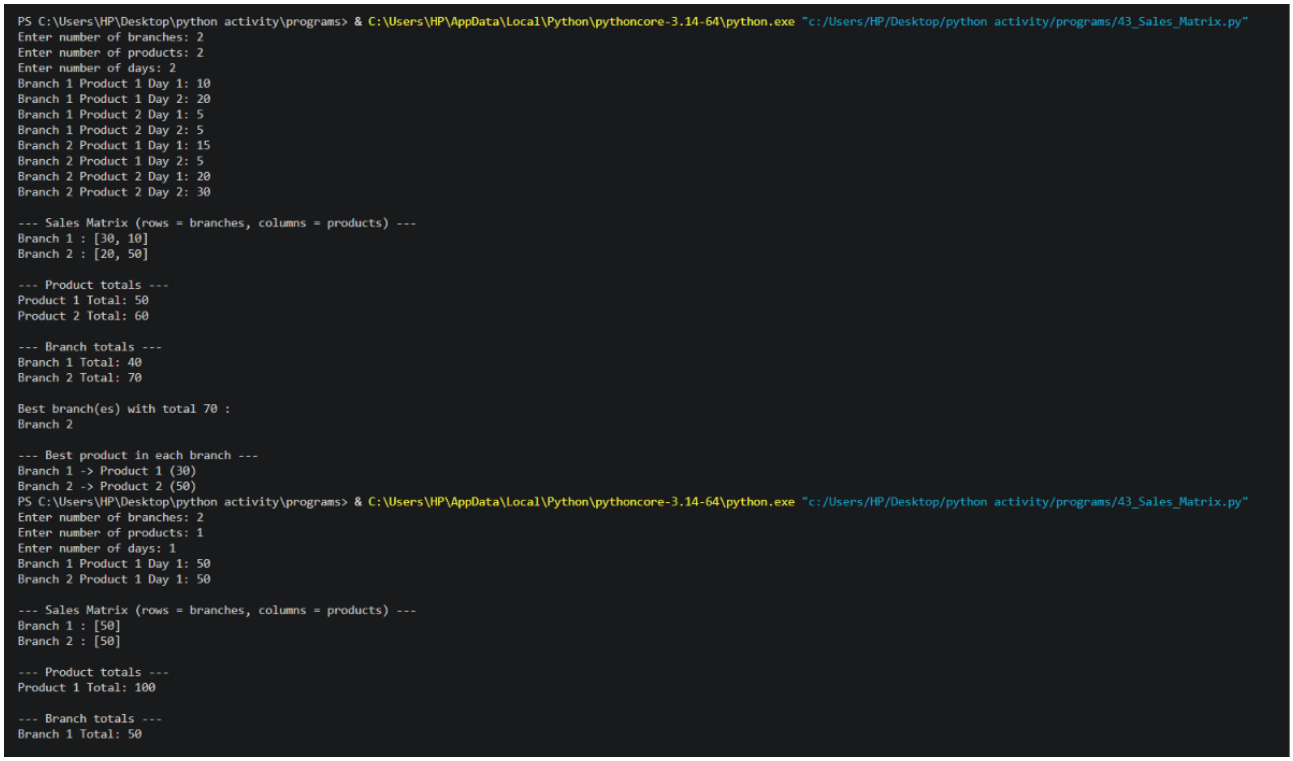
    # best product in each branch
    print("\n--- Best product in each branch ---")
    for i in range(b):
        best = 0
        for j in range(1, p):
            if sales[i][j] > sales[i][best]:
                best = j
        print("Branch", i + 1, "-> Product", best + 1, "(" + str(sales[i][best]) + ")")
```

8. Test Cases

• Test 1: 2 branches, 2 products, 2 days -> matrix [30, 10] and [20, 50]; product totals 50 and 60; branch totals 40 and 70; best branch: Branch 2

• Test 2: 2 branches, 1 product, 1 day, sales 50 and 50 -> both branches total 50, so Branch 1 and Branch 2 are the best

9. Output Screenshot(s)



10. Conclusion

The program compared branches and products using a nested list. Row-wise and column-wise traversal gave the totals and the best performers.

Case 48: Tuple-Based Employee Shift Conflict Detection

Python file: **48_Shift_Conflicts.py**

1. Problem Statement

Store employee shifts as tuples and detect overlapping shifts of the same employee.

2. Objectives

• Store each shift as a tuple (employee, start hour, end hour).
• Use tuple unpacking to read the values.
• Detect overlapping shifts of the same employee using nested loops.

3. Assumptions

• Start hour is between 0 and 23, end hour between 1 and 24, and start must be less than end.
• Only shifts of the same employee can conflict.
• A shift ending at 12 and another starting at 12 do not conflict.

4. Algorithm

1. Start.
2. Read the number of shifts n; it must be at least 2.
3. For each shift read the employee, start and end hour, validate, and store the tuple in a list.
4. Print all shifts using tuple unpacking.
5. Compare every pair of shifts (i, j) with j after i.
6. If both shifts belong to the same employee and start1 < end2 and start2 < end1, print a conflict.
7. If no conflict was found print None.
8. Stop.

5. Flowchart

START → Read n → n ≥ 2? → Read and store shifts as tuples → Print shifts → Compare every pair → Same employee and overlap? → Print conflict → None found? → STOP

6. Pseudocode

```
1. Read n. If n < 2: print error and stop.
2. For each shift read emp, start, end (start < end, 0 to 24); append (emp, start, end).
3. Print each shift by unpacking the tuple.
4. For i = 0 to n - 1: for j = i + 1 to n - 1:
5.    (e1, s1, f1) = shifts[i];  (e2, s2, f2) = shifts[j]
6.    If e1 = e2 and s1 < f2 and s2 < f1: print the conflict; found = true
7. If found is false print 'None'.
```

7. Python Source Code

```
# Tuple-Based Employee Shift Conflict Detection

n = int(input("Enter number of shifts: "))

if n < 2:
    print("Invalid! Enter at least 2 shifts.")
else:
    # store each shift as a tuple
    shifts = []
    for i in range(n):
        emp = input("Enter employee name: ")
        while True:
            start = int(input("Enter start hour (0-23): "))
            end = int(input("Enter end hour (1-24): "))
            if start >= 0 and end <= 24 and start < end:
                break
            print("Invalid! Start must be less than end, within 0 to 24.")
        shifts.append((emp, start, end))
```

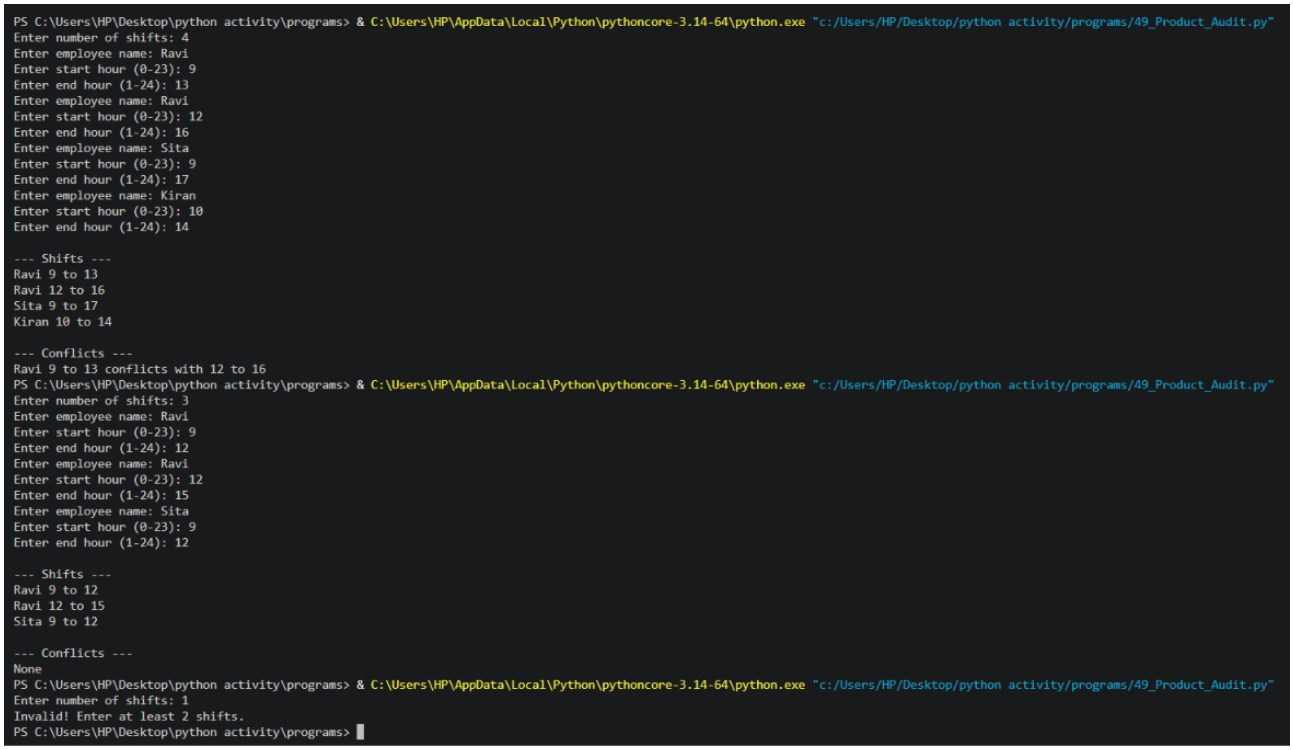
```
print("\n--- Shifts ---")
for emp, start, end in shifts:        # tuple unpacking
    print(emp, start, "to", end)

# compare every pair of shifts
print("\n--- Conflicts ---")
found = False
for i in range(n):
    for j in range(i + 1, n):
        e1, s1, f1 = shifts[i]
        e2, s2, f2 = shifts[j]
        if e1 == e2 and s1 < f2 and s2 < f1:
            print(e1, s1, "to", f1, "conflicts with", s2, "to", f2)
            found = True
if not found:
    print("None")
```

8. Test Cases

- Test 1: 4 shifts: Ravi 9-13, Ravi 12-16, Sita 9-17, Kiran 10-14 -> Ravi 9 to 13 conflicts with 12 to 16
- Test 2: 3 shifts: Ravi 9-12, Ravi 12-15, Sita 9-12 -> Conflicts: None
- Test 3: 1 shift -> Invalid! Enter at least 2 shifts.

9. Output Screenshot(s)



10. Conclusion

The program stored shifts as tuples and detected overlapping shifts of the same employee. It shows tuple creation, unpacking and nested loops for pair comparison.

Case 49: Tuple-Based Product Catalogue Audit

Python file: **49_Product_Audit.py**

1. Problem Statement

Store products as tuples, separate valid and invalid records, find the inventory value and list the products that need replenishment.

2. Objectives

• Store each product as a tuple (id, price, stock).
• Use tuple unpacking to check each record.
• Calculate the inventory value and find items with low stock.

3. Assumptions

• A record is invalid if the price is zero or negative, or the stock is negative.
• Stock 0 is valid (out of stock).
• A product needs replenishment when the stock is below 10.
• Inventory value = price x stock for valid records only.

4. Algorithm

1. Start.
2. Read the number of products n; it must be at least 1.
3. Read id, price and stock for each product and store the tuple in a list.
4. Unpack each tuple; if price <= 0 or stock < 0 move it to the invalid list, otherwise to the valid list.
5. For each valid product calculate price x stock and add it to the total inventory value.
6. Print the valid products and the total value.
7. Print the invalid records or None.
8. For valid products with stock below 10 print the quantity to order (10 - stock).
9. Stop.

5. Flowchart

START → Read n → n ≥ 1? → Read products as tuples → Unpack and separate valid / invalid → Inventory value → Print invalid records → Low stock items → STOP

6. Pseudocode

```
1. Read n. If n < 1: print error and stop.
2. For each product read pid, price, stock; append (pid, price, stock).
3. For each item: (pid, price, stock) = item.
4. If price <= 0 or stock < 0 add to invalid; else add to valid.
5. For each valid item: value = price * stock; total = total + value; print it.
6. Print invalid records, or 'None'.
7. For each valid item with stock < 10 print 'Order at least' 10 - stock.
```

7. Python Source Code

```
# Tuple-Based Product Catalogue Audit

REORDER_LEVEL = 10

n = int(input("Enter number of products: "))

if n < 1:
    print("Invalid! Number of products must be at least 1.")
else:
    # store each product as a tuple
    catalogue = []
    for i in range(n):
        pid = input("Enter product id: ")
```

```
        price = float(input("Enter price: "))
        stock = int(input("Enter stock: "))
        catalogue.append((pid, price, stock))

    # separate valid and invalid records
    valid = []
    invalid = []
    for item in catalogue:
        pid, price, stock = item          # tuple unpacking
        if price <= 0 or stock < 0:
            invalid.append(item)
        else:
            valid.append(item)

    # inventory value of valid records
    print("\n--- Valid products ---")
    total_value = 0
    for pid, price, stock in valid:
        value = price * stock
        total_value = total_value + value
        print(pid, "Price:", price, "Stock:", stock, "Value:", value)
    print("Total inventory value:", total_value)

    # invalid records
    print("\n--- Invalid records ---")
    if len(invalid) == 0:
        print("None")
    else:
        for pid, price, stock in invalid:
            print(pid, "Price:", price, "Stock:", stock)

    # replenishment
    print("\n--- Items needing replenishment (stock below 10) ---")
    need = False
    for pid, price, stock in valid:
        if stock < REORDER_LEVEL:
            print(pid, "Stock:", stock, "Order at least:", REORDER_LEVEL - stock)
            need = True
    if not need:
        print("None")
```

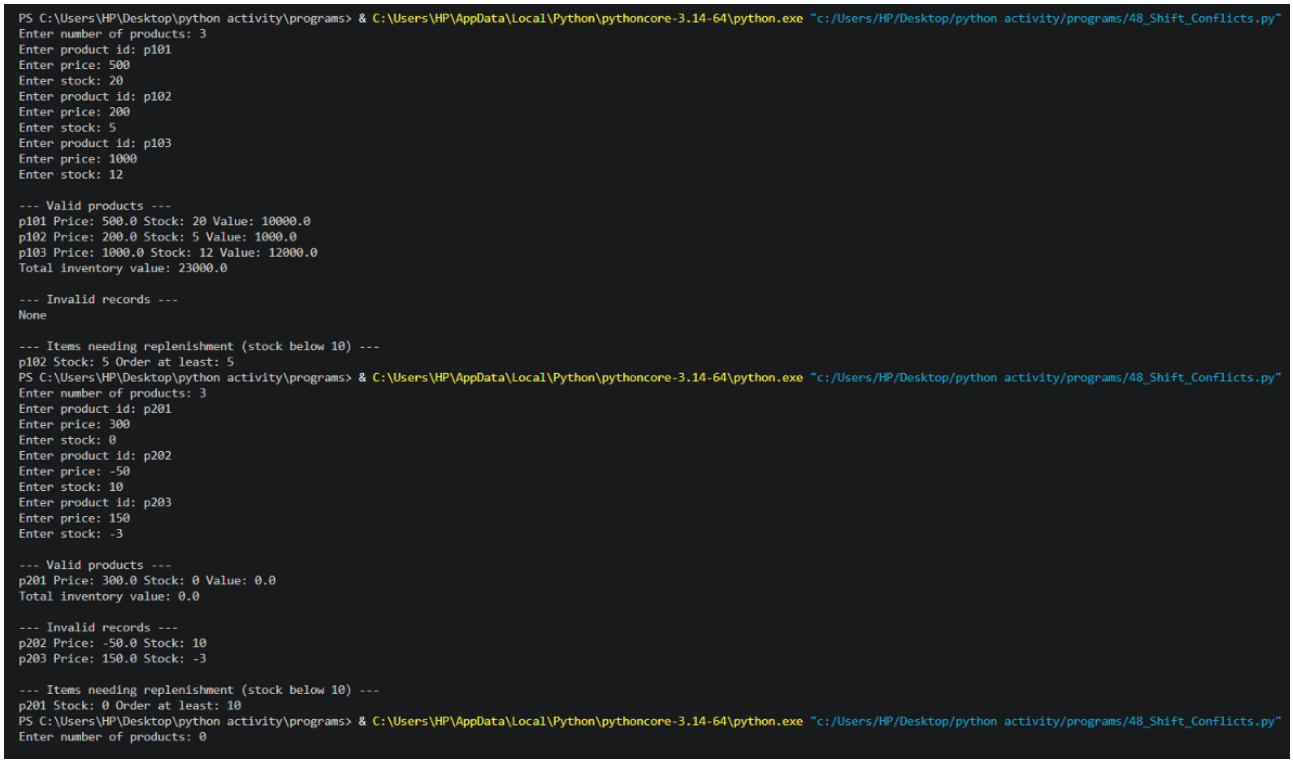
8. Test Cases

• Test 1: p101 (500, 20), p102 (200, 5), p103 (1000, 12) -> total inventory value 23000.0; no invalid records; p102 needs at least 5 more

• Test 2: p201 (300, 0), p202 (-50, 10), p203 (150, -3) -> p201 valid; p202 and p203 invalid; p201 needs at least 10

• Test 3: number of products 0 -> Invalid! Number of products must be at least 1.

9. Output Screenshot(s)



```
PS C:\Users\HP\Desktop\python activity\programs> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/HP/Desktop/python activity/programs/48_Shift_Conflicts.py"
Enter number of products: 3
Enter product id: p101
Enter price: 500
Enter stock: 20
Enter product id: p102
Enter price: 200
Enter stock: 5
Enter product id: p103
Enter price: 1000
Enter stock: 12

--- Valid products ---
p101 Price: 500.0 Stock: 20 Value: 10000.0
p102 Price: 200.0 Stock: 5 Value: 1000.0
p103 Price: 1000.0 Stock: 12 Value: 12000.0
Total inventory value: 23000.0

--- Invalid records ---
None

--- Items needing replenishment (stock below 10) ---
p102 Stock: 5 Order at least: 5
PS C:\Users\HP\Desktop\python activity\programs> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/HP/Desktop/python activity/programs/48_Shift_Conflicts.py"
Enter number of products: 3
Enter product id: p201
Enter price: 300
Enter stock: 0
Enter product id: p202
Enter price: -50
Enter stock: 10
Enter product id: p203
Enter price: 150
Enter stock: -3

--- Valid products ---
p201 Price: 300.0 Stock: 0 Value: 0.0
Total inventory value: 0.0

--- Invalid records ---
p202 Price: -50.0 Stock: 10
p203 Price: 150.0 Stock: -3

--- Items needing replenishment (stock below 10) ---
p201 Stock: 0 Order at least: 10
PS C:\Users\HP\Desktop\python activity\programs> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/HP/Desktop/python activity/programs/48_Shift_Conflicts.py"
Enter number of products: 0
```

10. Conclusion

The program audited a product catalogue using tuples, separated invalid records and reported the inventory value and low-stock items. It shows tuple unpacking in loops.